

Problem Statement

To develop a Customer Engagement Platform that enables businesses to engage with their customers more effectively through targeted messaging and real-time interactions.

Functional Requirements

1. Data Ingestion and Processing

- a. Data Ingestion to be happening for multiple sources including web, mobiles apps, other APIs and systems.
- b. Data ingestion should support real-time streaming and batch processing.
- c. AWS services like Amazon Kinesis, AWS Glue, and Amazon EMR will be used for data processing and transformation.

2. Warehousing and Dynamic Segmentation

- a. The data processed will be stored in Amazon Redshift (Data Warehouse) and AWS S3 (Data Lake), along with other data like orders and users related information.
- b. One thing to be noted is each of the events will have its own table and S3 prefix path.
- c. Segmentations are made via materialized views which provide an isolation between read and write throughputs which are refreshed via an orchestration tool like Airflow.
- d. Additionally services like Amazon EMR with Apache Spark can be incorporated to process historical data segments regarding the user behavior.

3. Campaign Management

- a. Campaigns should support personalized messages based on user segments and triggers.
- b. Amazon Lambda, Amazon RDS, Amazon SES, and Amazon SQS will be used for campaign management.
- c. RDS will contain campaign messages and the campaign ids.
- d. There will be a Checker Service which will check if the message is already sent to the user or not.

4. Analytics and Reporting

- a. The system should support real-time analytics for monitoring campaign performance, user engagement, and segmentation effectiveness.
- b. Analytics should include key metrics such as click-through rates, conversion rates, and user demographics.
- c. Amazon Redshift, BI tools like Amazon QuickSight, or Superset will be used for real-time analytics and reporting.

Non-Functional Requirements

1. Performance

- a. The system should support high throughput and low latency for data processing and query execution.
- b. Response times for campaign triggers and analytics queries should be within acceptable limits.

2. Reliability

- a. The system should be highly available and fault-tolerant.

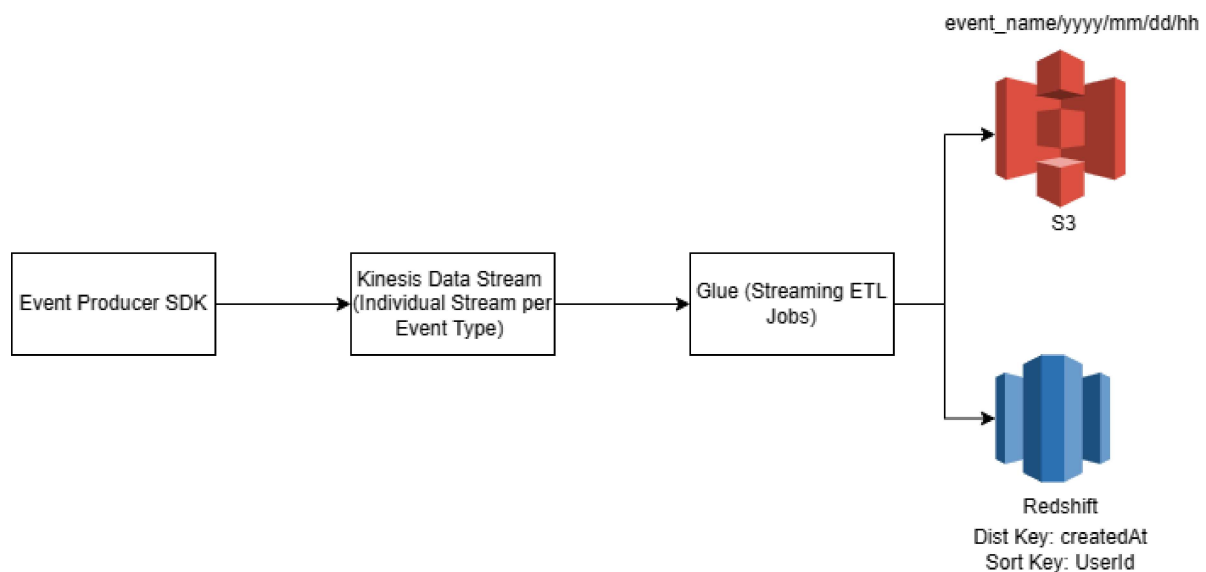
System Components and Architecture

Lego Blocks

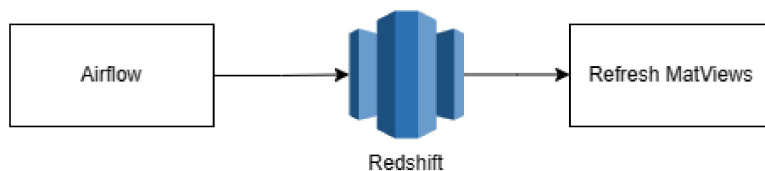
1. Event Streaming Layer: Kinesis DataStream and Kinesis Firehose
2. Data Processing Layer: AWS Glue, Airflow (Amazon EMR and Apache Spark)
3. Storage Layer: AWS S3, Amazon Redshift, Amazon RDS
4. Communications Layer: Amazon SQS and SES, AWS Lambda
5. Visualization: Superset (or) Amazon Quicksight
6. API Services: Amazon APIGateway

Workflow Diagram

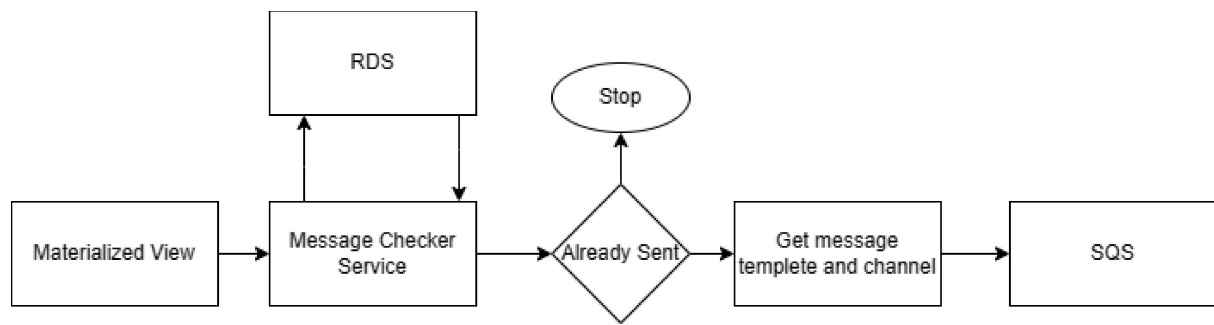
1. Data Streaming and Processing
 - a. Events to be stored in Redshift for a specific time period (say one month) to maintain queryable limits.
 - b. All the data will be stored in S3 for any processing use case beyond.
 - c. Assuming Redshift will have queryable aggregated data stored.



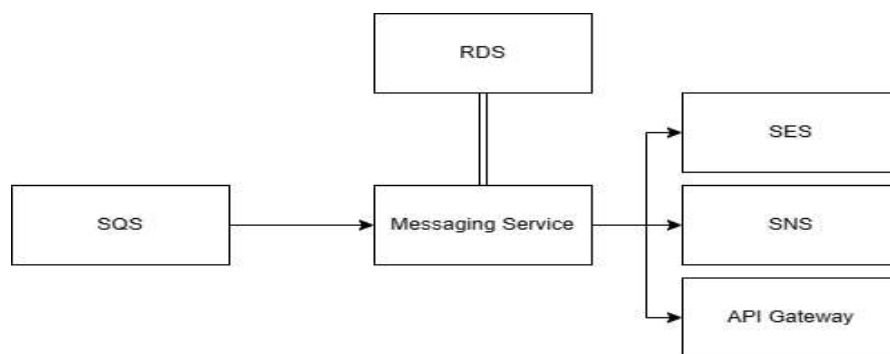
2. Materialized Views and Segmentation



3. Message Checker Service
 - a. This service is used to check if the message has been sent to the user.
 - b. If not, retrieve the message template and preferred communication channel.

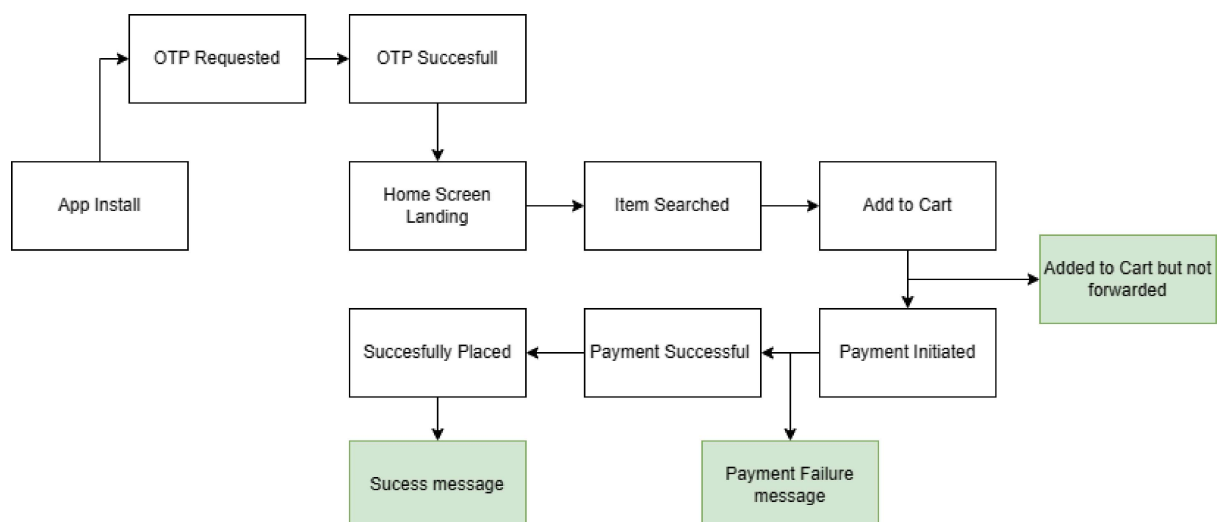


4. Communications



Example

Consider the below user journey as complete if the user completes his payment and successfully places his order. Let's say for example if the user abandons his cart midway and this is one of the use cases where the message nudge needs to be sent.



Say the events produced are UserHomeScreenLanding, ItemSearch, AddedToCart, PaymentInitiated, OrderSuccessful.

Now each of these tables will have their own S3 prefix paths and Redshift tables.

Materialized Views are created for this use-case: Like the users who have an event in AddedToCart but not in PaymentInitiated for this particular day. This view keeps on updating every 5 mins.

An RDS table with a campaign template to be sent for this use-case.

A campaign checker service which will check if the message is sent to this user or not; otherwise it will give the message template and preferred channel.

This is sent to a SQS for further execution for the message to be sent via notification service.

Assumptions

1. Emphasis is made more on data ingestion part and making it available to the end-user.
2. The checker service and messaging service are independent systems which can be developed extensively keeping features like rate limits etc.
3. The TRD assumes the implementation in terms of processing at every 5-10 mins frequency.
4. If needed for more real time, we can include things like DynamoDb and Lambda services process and send messages instantly by enabling **DynamoDB streams and triggering lambdas** based on it. .
5. The deliberate attempt to choose Redshift instead of DynamoDb is due to the fact that Redshift can support analytics as well as acting as a data warehouse for all the data processing jobs.
6. The better choice can be discussed and implemented.